

# LogiTrack

## Logistics and Delivery Analytics

*Final Database Project Report*

Rina Anuar, Khadizha Momunova, Ysmaiyl Apabek

# 1. Project Description

LogiTrack is a comprehensive relational database system designed to support analytics and operations management for a logistics and delivery company operating primarily in Kazakhstan and neighboring Central Asian countries. The system covers the full delivery lifecycle from shipment creation by a client, through warehouse dispatch and driver assignment, to final delivery and audit logging.

The project was selected from the LogiTrack Logistics and Delivery Analytics theme and implements a fully normalized, analytically capable database with realistic data volumes. It demonstrates advanced SQL techniques including window functions, conditional aggregation, nested subqueries, triggers, transactions, and index-driven optimization.

## 1.1 Business Context

The database models the core entities of a logistics operation:

- Geographic coverage: 60 cities across Kazakhstan, Russia, Kyrgyzstan, Uzbekistan, and Tajikistan.
- Infrastructure: 50 warehouses distributed across those cities, each with capacity and operational metadata.
- Fleet: 80 vehicles of 6 types, assigned to warehouses and tracked by year and activity status.
- Workforce: 70 drivers with ratings, hire dates, license numbers, and home warehouse assignments.
- Clients: 80 companies placing shipment orders, with contact details and city registrations.
- Routes: 80 defined origin-destination pairs with distance and estimated duration.
- Shipments: 350 shipment records linked to clients and warehouses, with weight, volume, and priority.
- Deliveries: 350 delivery records connecting all entities: driver, vehicle, route, warehouse with status, timing, cost, and delay tracking.
- Audit: `delivery_status_log` automatically records every status change via a trigger.

## 1.2 Technology Stack

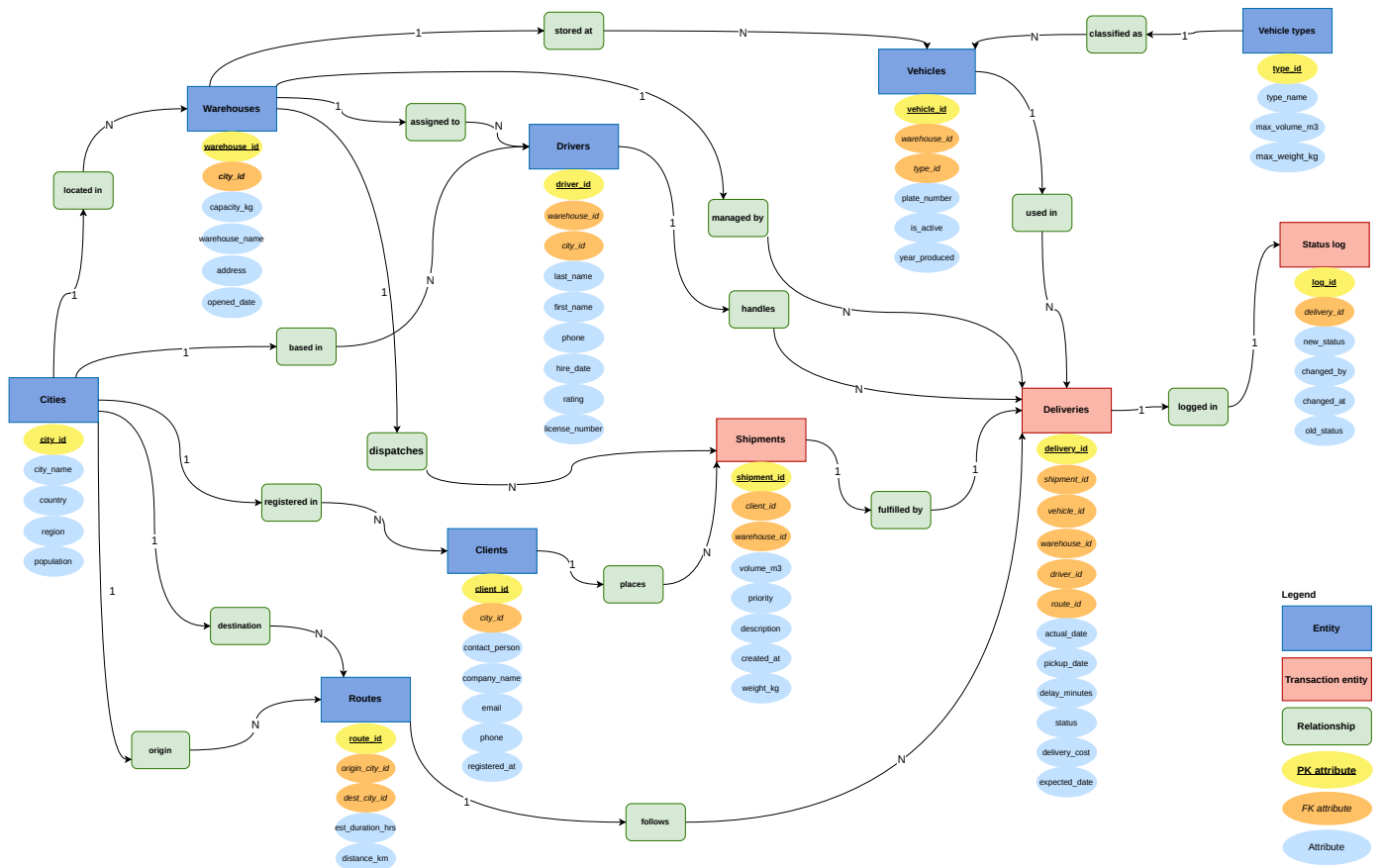
The database is implemented in MySQL 8.0 using InnoDB engine, utf8mb4 character set, and `unicode_ci` collation to support multi-language content. All scripts are modular and executable in sequence.

## 2. Entity-Relationship (ER) Diagram

The ER diagram below illustrates all 10 entities and 16 foreign-key relationships that form the LogiTrack schema. The diagram was designed in draw.io and is included as a separate PDF file in the project submission.

Key design decisions reflected in the diagram:

- Cities is a central reference entity connected to warehouses, drivers, clients, and routes (as both origin and destination).
- Deliveries is the main transactional entity at the center of the schema, referencing shipments, drivers, vehicles, routes, and warehouses.
- delivery\_status\_log is an audit entity connected only to deliveries, populated automatically by a trigger.
- A unique constraint on deliveries.shipment\_id enforces a strict 1:1 relationship between shipments and deliveries.



### 3. Database Schema

The schema consists of 10 tables organized into three logical layers:

| Layer              | Tables                                                                |
|--------------------|-----------------------------------------------------------------------|
| Reference / Lookup | cities, vehicle_types, vehicles, drivers, clients, warehouses, routes |
| Transactional      | shipments, deliveries                                                 |
| Audit              | delivery_status_log                                                   |

#### 3.1 Table Descriptions

##### cities

Stores geographic location data. Every city has a name, region, country, and population. A composite UNIQUE constraint prevents duplicate (city\_name, country) pairs.

| Column     | Type                  | Constraint                       | Description            |
|------------|-----------------------|----------------------------------|------------------------|
| city_id    | INT<br>AUTO_INCREMENT | PRIMARY KEY                      | Unique city identifier |
| city_name  | VARCHAR(100)          | NOT NULL                         | City name              |
| region     | VARCHAR(100)          | NOT NULL                         | Administrative region  |
| country    | VARCHAR(100)          | NOT NULL<br>DEFAULT 'Kazakhstan' | Country                |
| population | INT                   | NOT NULL,<br>CHECK > 0           | City population        |

##### warehouses

Represents physical warehouse locations. Each warehouse belongs to a city, has a name, address, storage capacity in kg, and opening date.

| Column         | Type                  | Constraint                       | Description               |
|----------------|-----------------------|----------------------------------|---------------------------|
| warehouse_id   | INT<br>AUTO_INCREMENT | PRIMARY KEY                      | Unique warehouse ID       |
| warehouse_name | VARCHAR(150)          | NOT NULL<br>UNIQUE               | Warehouse name            |
| city_id        | INT                   | FK → cities.city_id              | City location             |
| address        | VARCHAR(255)          | NOT NULL                         | Street address            |
| capacity_kg    | INT                   | NOT NULL,<br>CHECK > 0           | Max storage capacity (kg) |
| opened_date    | DATE                  | NOT NULL<br>DEFAULT CURRENT_DATE | Opening date              |

### vehicle\_types

Reference table for vehicle categories. Defines the maximum weight and volume each category can carry.

| Column        | Type                  | Constraint             | Description                        |
|---------------|-----------------------|------------------------|------------------------------------|
| type_id       | INT<br>AUTO_INCREMENT | PRIMARY KEY            | Unique type ID                     |
| type_name     | VARCHAR(50)           | NOT NULL<br>UNIQUE     | Type label (e.g. Large Truck)      |
| max_weight_kg | DECIMAL(10,2)         | NOT NULL,<br>CHECK > 0 | Max payload in kg                  |
| max_volume_m3 | DECIMAL(10,2)         | NOT NULL,<br>CHECK > 0 | Max cargo volume in m <sup>3</sup> |

### vehicles

Individual vehicles in the fleet. Each vehicle has a unique plate number, belongs to a type and a warehouse, and tracks its production year and active status.

| Column        | Type                  | Constraint                      | Description          |
|---------------|-----------------------|---------------------------------|----------------------|
| vehicle_id    | INT<br>AUTO_INCREMENT | PRIMARY KEY                     | Unique vehicle ID    |
| plate_number  | VARCHAR(20)           | NOT NULL UNIQUE                 | License plate        |
| type_id       | INT                   | FK →<br>vehicle_types.type_id   | Vehicle category     |
| warehouse_id  | INT                   | FK →<br>warehouses.warehouse_id | Home warehouse       |
| year_produced | INT                   | CHECK BETWEEN 1990<br>AND 2026  | Year of manufacture  |
| is_active     | BOOLEAN               | NOT NULL DEFAULT<br>TRUE        | Active/inactive flag |

### drivers

Driver records including personal details, contact info, license number, hire date, warehouse assignment, home city, and performance rating.

| Column         | Type                  | Constraint                       | Description            |
|----------------|-----------------------|----------------------------------|------------------------|
| driver_id      | INT<br>AUTO_INCREMENT | PRIMARY KEY                      | Unique driver ID       |
| first_name     | VARCHAR(50)           | NOT NULL                         | First name             |
| last_name      | VARCHAR(50)           | NOT NULL                         | Last name              |
| phone          | VARCHAR(20)           | NOT NULL UNIQUE                  | Contact phone          |
| license_number | VARCHAR(30)           | NOT NULL UNIQUE                  | Driving license number |
| hire_date      | DATE                  | NOT NULL DEFAULT<br>CURRENT_DATE | Date hired             |
| warehouse_id   | INT                   | FK →<br>warehouses.warehouse_id  | Assigned warehouse     |

|         |              |                         |                    |
|---------|--------------|-------------------------|--------------------|
| city_id | INT          | FK → cities.city_id     | Home city          |
| rating  | DECIMAL(3,2) | DEFAULT 5.00, CHECK 1–5 | Performance rating |

### clients

Companies that place shipment orders. Each client has a company name, contact person, unique email and phone, and is registered in a city.

| Column         | Type                  | Constraint                   | Description       |
|----------------|-----------------------|------------------------------|-------------------|
| client_id      | INT<br>AUTO_INCREMENT | PRIMARY KEY                  | Unique client ID  |
| company_name   | VARCHAR(200)          | NOT NULL                     | Company name      |
| contact_person | VARCHAR(100)          |                              | Contact name      |
| email          | VARCHAR(150)          | NOT NULL UNIQUE              | Business email    |
| phone          | VARCHAR(20)           |                              | Contact phone     |
| city_id        | INT                   | FK → cities.city_id          | Client city       |
| registered_at  | TIMESTAMP             | DEFAULT<br>CURRENT_TIMESTAMP | Registration time |

### routes

Predefined origin-destination pairs with distance and estimated travel time. A composite unique constraint prevents duplicate routes, and a CHECK constraint ensures origin and destination differ.

| Column             | Type                  | Constraint             | Description                 |
|--------------------|-----------------------|------------------------|-----------------------------|
| route_id           | INT<br>AUTO_INCREMENT | PRIMARY KEY            | Unique route ID             |
| origin_city_id     | INT                   | FK → cities.city_id    | Departure city              |
| dest_city_id       | INT                   | FK → cities.city_id    | Destination city            |
| distance_km        | DECIMAL(10,2)         | NOT NULL,<br>CHECK > 0 | Route distance (km)         |
| est_duration_hours | DECIMAL(6,2)          | NOT NULL,<br>CHECK > 0 | Estimated travel time (hrs) |

### shipments

Shipment orders created by clients. Each shipment is associated with a client, dispatched from a warehouse, and has weight, volume, description, and priority.

| Column       | Type                  | Constraint                      | Description          |
|--------------|-----------------------|---------------------------------|----------------------|
| shipment_id  | INT<br>AUTO_INCREMENT | PRIMARY KEY                     | Unique shipment ID   |
| client_id    | INT                   | FK → clients.client_id          | Ordering client      |
| warehouse_id | INT                   | FK →<br>warehouses.warehouse_id | Dispatch warehouse   |
| weight_kg    | DECIMAL(10,2)         | NOT NULL, CHECK > 0             | Shipment weight (kg) |

|             |               |                                      |                      |
|-------------|---------------|--------------------------------------|----------------------|
| volume_m3   | DECIMAL(10,2) | NOT NULL, CHECK > 0                  | Cargo volume (m³)    |
| description | VARCHAR(255)  |                                      | Contents description |
| created_at  | TIMESTAMP     | DEFAULT CURRENT_TIMESTAMP            | Order timestamp      |
| priority    | VARCHAR(10)   | CHECK IN (low, normal, high, urgent) | Delivery priority    |

### deliveries

The central transactional table. Each delivery links a shipment to a driver, vehicle, route, and warehouse. It tracks status, scheduled and actual dates, delay in minutes, and delivery cost. A UNIQUE constraint on shipment\_id enforces the 1:1 relationship between shipments and deliveries.

| Column        | Type                  | Constraint                                                 | Description               |
|---------------|-----------------------|------------------------------------------------------------|---------------------------|
| delivery_id   | INT<br>AUTO_INCREMENT | PRIMARY KEY                                                | Unique delivery ID        |
| shipment_id   | INT                   | UNIQUE, FK → shipments                                     | One shipment per delivery |
| driver_id     | INT                   | FK → drivers.driver_id                                     | Assigned driver           |
| vehicle_id    | INT                   | FK → vehicles.vehicle_id                                   | Assigned vehicle          |
| route_id      | INT                   | FK → routes.route_id                                       | Selected route            |
| warehouse_id  | INT                   | FK → warehouses.warehouse_id                               | Dispatching warehouse     |
| status        | VARCHAR(20)           | CHECK IN (pending, in_transit, delivered, delayed, failed) | Delivery status           |
| pickup_date   | DATETIME              | NOT NULL                                                   | Actual pickup time        |
| expected_date | DATETIME              | NOT NULL, CHECK > pickup_date                              | Expected delivery time    |
| actual_date   | DATETIME              | NULL allowed                                               | Actual delivery time      |
| delay_minutes | INT                   | DEFAULT 0, CHECK >= 0                                      | Delay in minutes          |
| delivery_cost | DECIMAL(12,2)         | NOT NULL, CHECK > 0                                        | Delivery fee (KZT)        |

### delivery\_status\_log

Audit table that records every status transition for every delivery. Populated automatically by the trigger trg\_delivery\_status\_audit.

| Column | Type                  | Constraint  | Description         |
|--------|-----------------------|-------------|---------------------|
| log_id | INT<br>AUTO_INCREMENT | PRIMARY KEY | Unique log entry ID |

|             |             |                                |                        |
|-------------|-------------|--------------------------------|------------------------|
| delivery_id | INT         | FK →<br>deliveries.delivery_id | Delivery being tracked |
| old_status  | VARCHAR(20) | NULL allowed                   | Previous status        |
| new_status  | VARCHAR(20) | NOT NULL                       | New status applied     |
| changed_at  | TIMESTAMP   | DEFAULT<br>CURRENT_TIMESTAMP   | Time of change         |
| changed_by  | VARCHAR(50) | DEFAULT 'system'               | User or system actor   |



## 4. Normalization (0NF → 3NF)

This section traces the normalization journey of the LogiTrack data model from an unnormalized flat table to a fully Third Normal Form (3NF) schema. Each step identifies the problems present and the fixes applied.

### 4.1 0NF — Unnormalized Form

The starting point is a single table `DeliveryLog_0NF` containing all delivery information. This structure has severe problems:

- Multi-valued attributes: `shipment_ids`, `warehouse_names`, `delivery_statuses`, and `client_names` all store comma-separated lists in a single cell.
- Repeating groups: `vehicle_types` and `route_cities` hold multiple values per row.
- Data redundancy: driver name and phone appear multiple times for the same driver.
- Update anomalies: changing a driver's phone number requires updating every row that mentions that driver.
- Insertion anomalies: a new warehouse cannot be added without a delivery.
- Deletion anomalies: deleting the only delivery from a warehouse loses the warehouse data.

Example 0NF row:

| delivery_id | shipment_ids           | driver_name    | driver_phone | warehouse_names                    | warehouse_citi... | vehicle_typ... | route_cities                          | delivery_statuses        | client_names                    |
|-------------|------------------------|----------------|--------------|------------------------------------|-------------------|----------------|---------------------------------------|--------------------------|---------------------------------|
| 1           | SHP001, SHP002         | Arman Sadykov  | +77771234567 | Almaty Central WH, Astana North WH | Almaty, Astana    | Truck, Van     | Almaty → Astana, Astana → Karaganda   | On Time, Delayed         | Kaspi Delivery, DHL Kazakhstan  |
| 2           | SHP003                 | Dias Nurpeisov | +77775554433 | Shymkent South WH                  | Shymkent          | Truck          | Shymkent → Taraz                      | On Time                  | Wildberries                     |
| 3           | SHP004, SHP005, SHP006 | Arman Sadykov  | +77771234567 | Almaty Central WH                  | Almaty            | Truck, Sedan   | Almaty → Taldykorgan, Almaty → Astana | Failed, On Time, Delayed | Ozon Kazakhstan, Kaspi Delivery |

### 4.2 1NF — First Normal Form

To achieve 1NF, every cell must contain an atomic (single) value, and repeating groups must be removed.

Fixes applied:

- Each shipment is stored as a separate row.
- All multi-valued columns are split: one value per column per row.
- A composite primary key (`delivery_id`, `shipment_id`) uniquely identifies each row.

Remaining problems after 1NF:

- Driver information (name, phone) is still duplicated across many rows — `driver_phone` depends only on `driver_name`, not on the full key.
- Warehouse information is duplicated similarly.
- These are partial dependencies — a violation of 2NF.

| delivery_id | shipment_id | driver_name    | driver_phone | warehouse_name    | warehouse_city | vehicle_type | route_city           | delivery_stat... | client_name     |
|-------------|-------------|----------------|--------------|-------------------|----------------|--------------|----------------------|------------------|-----------------|
| 1           | SHP001      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Truck        | Almaty → Astana      | On Time          | Kaspi Delivery  |
| 1           | SHP002      | Arman Sadykov  | +77771234567 | Astana North WH   | Astana         | Van          | Astana → Karaganda   | Delayed          | DHL Kazakhstan  |
| 2           | SHP003      | Dias Nurpeisov | +77775554433 | Shymkent South WH | Shymkent       | Truck        | Shymkent → Taraz     | On Time          | Wildberries     |
| 3           | SHP004      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Truck        | Almaty → Taldykorgan | Failed           | Ozon Kazakhstan |
| 3           | SHP005      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Sedan        | Almaty → Astana      | On Time          | Kaspi Delivery  |
| 3           | SHP006      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Sedan        | Almaty → Astana      | Delayed          | Kaspi Delivery  |

### 4.3 2NF — Second Normal Form

To achieve 2NF, every non-key attribute must depend on the whole primary key, not just part of it.

Fixes applied:

- Drivers\_2NF created: driver\_id, driver\_name, driver\_phone — driver details depend only on driver\_id.
- Warehouses\_2NF created: warehouse\_id, warehouse\_name, warehouse\_city.
- Deliveries\_2NF now references driver\_id and warehouse\_id as foreign keys instead of duplicating the text.

Remaining problems after 2NF:

- Transitive dependency: warehouse\_id → warehouse\_city (city depends on the warehouse, not directly on the delivery).
- vehicle\_type is still stored as free text in many rows, creating redundancy.
- These are transitive dependencies — a violation of 3NF.

| delivery_id | shipment_id | driver_name    | driver_phone | warehouse_name    | warehouse_city | vehicle_type | route_city           | delivery_stat... | client_name     |
|-------------|-------------|----------------|--------------|-------------------|----------------|--------------|----------------------|------------------|-----------------|
| 1           | SHP001      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Truck        | Almaty → Astana      | On Time          | Kaspi Delivery  |
| 1           | SHP002      | Arman Sadykov  | +77771234567 | Astana North WH   | Astana         | Van          | Astana → Karaganda   | Delayed          | DHL Kazakhstan  |
| 2           | SHP003      | Dias Nurpeisov | +77775554433 | Shymkent South WH | Shymkent       | Truck        | Shymkent → Taraz     | On Time          | Wildberries     |
| 3           | SHP004      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Truck        | Almaty → Taldykorgan | Failed           | Ozon Kazakhstan |
| 3           | SHP005      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Sedan        | Almaty → Astana      | On Time          | Kaspi Delivery  |
| 3           | SHP006      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Sedan        | Almaty → Astana      | Delayed          | Kaspi Delivery  |

### 4.4 3NF — Third Normal Form

To achieve 3NF, every non-key attribute must depend directly and only on the primary key — no transitive dependencies.

Fixes applied:

- Cities\_3NF created: city information extracted so warehouses reference city\_id instead of storing city\_name text.
- VehicleTypes\_3NF created: vehicle type names stored once; deliveries reference vehicle\_type\_id.
- Routes\_3NF created: route distances stored once and referenced from deliveries.
- Clients\_3NF created: client names stored once.

Final 3NF state: Every non-key attribute in every table depends on the primary key, the whole primary key, and nothing but the primary key. The schema eliminates redundancy, update anomalies, insertion anomalies, and deletion anomalies.

| delivery_id | shipment_id | driver_name    | driver_phone | warehouse_name    | warehouse_city | vehicle_type | route                | distance_km | client_name     | delivery_stat... |
|-------------|-------------|----------------|--------------|-------------------|----------------|--------------|----------------------|-------------|-----------------|------------------|
| 1           | SHP001      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Truck        | Almaty → Astana      | 1200.00     | Kaspi Delivery  | On Time          |
| 1           | SHP002      | Arman Sadykov  | +77771234567 | Astana North WH   | Astana         | Van          | Astana → Karaganda   | 200.00      | DHL Kazakhstan  | Delayed          |
| 2           | SHP003      | Dias Nurpeisov | +77775554433 | Shymkent South WH | Shymkent       | Truck        | Shymkent → Taraz     | 180.00      | Wildberries     | On Time          |
| 3           | SHP004      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Truck        | Almaty → Taldykorgan | 300.00      | Ozon Kazakhstan | Failed           |
| 3           | SHP005      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Sedan        | Almaty → Astana      | 1200.00     | Kaspi Delivery  | On Time          |
| 3           | SHP006      | Arman Sadykov  | +77771234567 | Almaty Central WH | Almaty         | Sedan        | Almaty → Astana      | 1200.00     | Kaspi Delivery  | Delayed          |

| Normal Form | Requirement                              | Status in LogiTrack                                 |
|-------------|------------------------------------------|-----------------------------------------------------|
| 1NF         | Atomic values, no repeating groups       | All cells atomic, one value per column per row      |
| 2NF         | No partial dependencies on composite key | All non-key columns depend on full PK               |
| 3NF         | No transitive dependencies               | No non-key column depends on another non-key column |

## 5. Constraints and Relationships

### 5.1 Primary Keys

Every table uses a single-column surrogate INT AUTO\_INCREMENT primary key named <entity>\_id. This provides stable, compact join keys with no business-logic coupling.

### 5.2 Foreign Keys (16 total)

The 16 foreign-key relationships enforce referential integrity across all tables:

| Table               | FK Column                                                              | References                   | Count |
|---------------------|------------------------------------------------------------------------|------------------------------|-------|
| warehouses          | city_id                                                                | cities.city_id               | 1     |
| vehicles            | type_id,<br>warehouse_id                                               | vehicle_types,<br>warehouses | 2     |
| drivers             | warehouse_id,<br>city_id                                               | warehouses, cities           | 2     |
| clients             | city_id                                                                | cities.city_id               | 1     |
| routes              | origin_city_id,<br>dest_city_id                                        | cities (twice)               | 2     |
| shipments           | client_id,<br>warehouse_id                                             | clients, warehouses          | 2     |
| deliveries          | shipment_id,<br>driver_id,<br>vehicle_id,<br>route_id,<br>warehouse_id | 5 tables                     | 5     |
| delivery_status_log | delivery_id                                                            | deliveries.delivery_id       | 1     |

### 5.3 UNIQUE Constraints

- cities: (city\_name, country) — prevents duplicate city-country pairs.
- warehouses: warehouse\_name — each warehouse has a unique name.
- vehicles: plate\_number — unique license plate per vehicle.
- drivers: phone, license\_number — unique contact and license per driver.
- clients: email — unique business email per client.
- routes: (origin\_city\_id, dest\_city\_id) — only one route definition per direction.
- deliveries: shipment\_id — enforces the 1:1 shipment-to-delivery rule.

### 5.4 NOT NULL Constraints

All columns where the business logic requires a value are marked NOT NULL. This includes all names, amounts, dates, and foreign keys. Optional fields (description, contact\_person, actual\_date) allow NULL where appropriate.

### 5.5 DEFAULT Values

- cities.country = 'Kazakhstan' — most data is domestic.

- warehouses.opened\_date = CURRENT\_DATE — defaults to today if not specified.
- drivers.hire\_date = CURRENT\_DATE, rating = 5.00 — new hires start with full score.
- clients.registered\_at = CURRENT\_TIMESTAMP — automatic registration timestamp.
- deliveries.status = 'pending', delay\_minutes = 0 — sensible starting state.
- delivery\_status\_log.changed\_by = 'system' — trigger-populated rows default to system.

## 5.6 CHECK Constraints

All business-rule numeric and enum constraints are enforced with CHECK:

- cities: population > 0
- warehouses: capacity\_kg > 0
- vehicle\_types: max\_weight\_kg > 0, max\_volume\_m3 > 0
- vehicles: year\_produced BETWEEN 1990 AND 2026
- drivers: rating BETWEEN 1.00 AND 5.00
- routes: distance\_km > 0, est\_duration\_hours > 0, origin\_city\_id <> dest\_city\_id
- shipments: weight\_kg > 0, volume\_m3 > 0, priority IN ('low','normal','high','urgent')
- deliveries: status IN ('pending','in\_transit','delivered','delayed','failed'), expected\_date > pickup\_date, actual\_date IS NULL OR actual\_date >= pickup\_date, delay\_minutes >= 0, delivery\_cost > 0

## 6. Analytical SQL Queries

Six analytical queries are implemented, each demonstrating a different advanced SQL technique as required by the course specification. Outputs for each query are shown in the project defense.

### Query 1 — Top 5 Drivers by On-Time Deliveries

Purpose: Identify the five best-performing drivers by counting perfectly on-time deliveries (status = 'delivered', delay = 0, actual\_date present) and calculating their average delivery duration in hours.

Techniques demonstrated: Multi-table JOIN, aggregation with COUNT and AVG, TIMESTAMPDIF for duration calculation, WHERE filter, GROUP BY, ORDER BY with two criteria, LIMIT.

```
-- -----  
-- QUERY 1  
-- Top 5 drivers with highest number of on-time deliveries  
-- and their average delivery duration (in hours).  
-- Demonstrates: JOIN, aggregation, filtering, ORDER BY + LIMIT.  
-- -----  
  
SELECT  
    d.driver_id,  
    CONCAT(d.first_name, ' ', d.last_name) AS driver_name,  
    COUNT(*) AS on_time_deliveries,  
    ROUND(AVG(TIMESTAMPDIFF(MINUTE, dl.pickup_date, dl.actual_date) / 60.0), 2) AS avg_duration_hours  
FROM deliveries dl  
JOIN drivers d ON d.driver_id = dl.driver_id  
WHERE dl.status = 'delivered'  
    AND dl.delay_minutes = 0  
    AND dl.actual_date IS NOT NULL  
GROUP BY d.driver_id, d.first_name, d.last_name  
ORDER BY on_time_deliveries DESC, avg_duration_hours ASC  
LIMIT 5;
```

[INSERT QUERY 1 OUTPUT TABLE]

| driver_id | driver_name       | on_time_deliveries | avg_duration_hours |
|-----------|-------------------|--------------------|--------------------|
| 1         | Aibek Nursultanov | 5                  | 16.00              |
| 16        | Farid Kenzhebaev  | 5                  | 16.00              |
| 3         | Erlan Smagulev    | 5                  | 16.00              |
| 19        | Timur Akhmetov    | 5                  | 16.00              |
| 12        | Bolat Nurpeisov   | 5                  | 16.00              |

### Query 2 — Warehouses Above Average Delay

Purpose: Find warehouses whose average delivery delay exceeds the company-wide average. This helps management identify underperforming dispatch locations.

Techniques demonstrated: JOIN with aggregation, nested subquery in HAVING clause comparing group aggregate to a global aggregate.

```
-----  
-- QUERY 2  
-- Warehouses whose AVERAGE delivery delay is higher than the  
-- overall company-wide average delay.  
-- Demonstrates: nested subquery in HAVING, aggregation, JOIN.  
-----  
  
SELECT  
    w.warehouse_id,  
    w.warehouse_name,  
    ROUND(AVG(dl.delay_minutes), 2) AS avg_delay_minutes,  
    COUNT(*)                       AS total_deliveries  
FROM deliveries dl  
JOIN warehouses w ON w.warehouse_id = dl.warehouse_id  
GROUP BY w.warehouse_id, w.warehouse_name  
) HAVING AVG(dl.delay_minutes) > (  
    SELECT AVG(delay_minutes) FROM deliveries  
- )  
ORDER BY avg_delay_minutes DESC;
```

[INSERT QUERY 2 OUTPUT TABLE]

| warehouse_id | warehouse_name          | avg_delay_minutes | total_deliveries |
|--------------|-------------------------|-------------------|------------------|
| 2            | Almaty South Depot      | 120.00            | 7                |
| 49           | Samarkand Partner Depot | 120.00            | 7                |
| 4            | Astana East Depot       | 120.00            | 7                |
| 6            | Karaganda Depot         | 120.00            | 7                |
| 7            | Aktobe Warehouse        | 120.00            | 7                |
| 8            | Taraz Storage           | 120.00            | 7                |
| 11           | Semey Warehouse         | 120.00            | 7                |
| 12           | Atyrau Oil Depot        | 120.00            | 7                |
| 13           | Kostanay Agro Hub       | 120.00            | 7                |
| 15           | Oral West Hub           | 120.00            | 7                |
| 16           | Aktau Port Warehouse    | 120.00            | 7                |

|    |                             |        |   |
|----|-----------------------------|--------|---|
| 21 | Rudny Metal Depot           | 120.00 | 7 |
| 24 | Balkhash Copper Hub         | 120.00 | 7 |
| 25 | Kentau Depot                | 120.00 | 7 |
| 3  | Astana Main Warehouse       | 120.00 | 7 |
| 29 | Kyzylorda Depot             | 120.00 | 7 |
| 30 | Bishkek Partner Hub         | 120.00 | 7 |
| 31 | Tashkent International D... | 120.00 | 7 |
| 33 | Omsk Depot                  | 120.00 | 7 |
| 35 | Moscow Partner Depot        | 120.00 | 7 |
| 38 | Shymkent North Depot        | 120.00 | 7 |
| 39 | Atyrau Transit Hub          | 120.00 | 7 |
| 42 | Dostyk Rail Depot           | 120.00 | 7 |
| 43 | Beineu Transit              | 120.00 | 7 |
| 47 | Almaty Tech Park Hub        | 120.00 | 7 |
| 48 | Astana Expo Depot           | 120.00 | 7 |
| 9  | Pavlodar Depot              | 102.86 | 7 |
| 17 | Taldykorgan Depot           | 102.86 | 7 |
| 18 | Kokshetau Storage           | 102.86 | 7 |
| 20 | Ekibastuz Power Hub         | 102.86 | 7 |
| 34 | Khorgos FTZ Warehouse       | 102.86 | 7 |
| 36 | Almaty Airport Cargo        | 102.86 | 7 |
| 40 | Kapchagay Depot             | 102.86 | 7 |
| 45 | Aralsk Depot                | 102.86 | 7 |
| 27 | Saran Depot                 | 102.86 | 7 |

### Query 3 — Routes Where Delayed > On-Time

Purpose: Identify problematic routes where the number of delayed deliveries exceeds the number of on-time deliveries. These routes may need operational review.

Techniques demonstrated: Conditional aggregation using CASE WHEN inside SUM, joining cities twice (once for origin, once for destination), HAVING comparing two aggregated expressions.



```

-----
-- QUERY 3
-- Routes where the number of DELAYED deliveries is greater than
-- the number of ON-TIME deliveries.
-- Demonstrates: conditional aggregation with CASE inside SUM,
-- HAVING with comparison of two aggregates, JOIN to cities.
-----

SELECT
    r.route_id,
    co.city_name          AS origin_city,
    cd.city_name          AS dest_city,
    SUM(CASE WHEN dl.status = 'delayed' THEN 1 ELSE 0 END) AS delayed_count,
)  SUM(CASE WHEN dl.status = 'delivered'
-    AND dl.delay_minutes = 0 THEN 1 ELSE 0 END) AS on_time_count
FROM deliveries dl
JOIN routes r ON r.route_id = dl.route_id
JOIN cities co ON co.city_id = r.origin_city_id
JOIN cities cd ON cd.city_id = r.dest_city_id
GROUP BY r.route_id, co.city_name, cd.city_name
HAVING delayed_count > on_time_count
ORDER BY delayed_count DESC;

```

[INSERT QUERY 3 OUTPUT TABLE]

| route_id | origin_city | dest_city | delayed_count | on_time_count |
|----------|-------------|-----------|---------------|---------------|
| 56       | Shu         | Taraz     | 2             | 1             |

## Query 4 — Rank Cities by Total Shipment Volume (Window Function)

Purpose: Rank all cities by the total volume of shipments dispatched from their warehouses. Useful for capacity planning and regional investment decisions.

Techniques demonstrated: Window function RANK() OVER (ORDER BY ...) applied after GROUP BY aggregation, three-table JOIN chain.

```

-----
-- QUERY 4
-- Rank cities (as shipment ORIGINS) by total shipment volume
-- using a WINDOW FUNCTION.
-- Demonstrates: window function RANK() OVER (...), JOINS.
-----

```

```

SELECT
    c.city_id,
    c.city_name,
    ROUND(SUM(s.volume_m3), 2)                                AS total_volume_m3,
    RANK() OVER (ORDER BY SUM(s.volume_m3) DESC)              AS volume_rank
FROM shipments s
JOIN warehouses w ON w.warehouse_id = s.warehouse_id
JOIN cities      c ON c.city_id      = w.city_id
GROUP BY c.city_id, c.city_name
ORDER BY volume_rank;

```

[INSERT QUERY 4 OUTPUT TABLE]

| city_id | city_name     | total_volume_m3 | volume_rank |
|---------|---------------|-----------------|-------------|
| 2       | Astana        | 586.00          | 1           |
| 1       | Almaty        | 580.00          | 2           |
| 3       | Shymkent      | 270.00          | 3           |
| 10      | Atyrau        | 238.00          | 4           |
| 30      | Zhezkazgan    | 171.50          | 5           |
| 53      | Beineu        | 166.50          | 6           |
| 58      | Saryagash     | 164.50          | 7           |
| 16      | Kokshetau     | 161.50          | 8           |
| 14      | Aktau         | 159.50          | 9           |
| 12      | Petropavlovsk | 157.50          | 10          |
| 35      | Omsk          | 156.50          | 11          |
| 33      | Tashkent      | 154.50          | 12          |
| 56      | Kyzylorda     | 152.50          | 13          |
| 6       | Taraz         | 151.50          | 14          |
| 25      | Saran         | 150.50          | 15          |
| 60      | Kandyagash    | 149.50          | 16          |
| 4       | Karaganda     | 149.50          | 16          |
| 54      | Shu           | 147.50          | 18          |
| 21      | Zhanaozen     | 146.50          | 19          |
| 52      | Dostyk        | 145.50          | 20          |
| 19      | Rudny         | 144.50          | 21          |
| 57      | Kapchagay     | 143.50          | 22          |
| 17      | Turkistan     | 142.50          | 23          |
| 15      | Taldykorgan   | 140.50          | 24          |
| 13      | Oral          | 138.50          | 25          |

|    |               |        |    |
|----|---------------|--------|----|
| 51 | Khorgos       | 137.50 | 26 |
| 11 | Kostanay      | 136.50 | 27 |
| 34 | Novosibirsk   | 135.50 | 28 |
| 9  | Semey         | 134.50 | 29 |
| 32 | Bishkek       | 133.50 | 30 |
| 7  | Pavlodar      | 132.50 | 31 |
| 37 | Samarkand     | 132.50 | 31 |
| 5  | Aktobe        | 130.50 | 33 |
| 24 | Ridder        | 129.50 | 34 |
| 55 | Aralsk        | 128.50 | 35 |
| 22 | Balkhash      | 127.50 | 36 |
| 20 | Temirtau      | 125.50 | 37 |
| 18 | Ekibastuz     | 123.50 | 38 |
| 31 | Moscow        | 118.50 | 39 |
| 50 | Kazan         | 113.50 | 40 |
| 8  | Ust-Kameno... | 113.50 | 40 |
| 23 | Kentau        | 108.50 | 42 |

## Query 5 — Classify Deliveries by Performance (CASE WHEN)

Purpose: Categorize every delivery into one of four performance buckets for SLA reporting and analytics dashboards.

Techniques demonstrated: Multi-branch CASE WHEN analytical logic for classification, GROUP BY on a derived expression, COUNT and AVG per category.

```

-----
-- QUERY 5
-- Classify deliveries by performance using CASE WHEN:
--   On Time      : status delivered AND delay = 0
--   Slightly Delayed : delay between 1 and 180 minutes
--   Seriously Delayed : delay > 180 minutes
--   Failed       : status failed
-- Demonstrates: CASE WHEN analytical logic, GROUP BY.
-----

SELECT
  CASE
    WHEN status = 'failed' THEN 'Failed'
    WHEN status = 'delivered' AND delay_minutes = 0 THEN 'On Time'
    WHEN delay_minutes BETWEEN 1 AND 180 THEN 'Slightly Delayed'
    WHEN delay_minutes > 180 THEN 'Seriously Delayed'
    ELSE 'Other'
  END AS performance_category,
  COUNT(*) AS delivery_count,
  ROUND(AVG(delay_minutes), 2) AS avg_delay
FROM deliveries
GROUP BY performance_category
ORDER BY delivery_count DESC;

```

[INSERT QUERY 5 OUTPUT TABLE]

| performance_category | delivery_count | avg_delay |
|----------------------|----------------|-----------|
| On Time              | 238            | 0.00      |
| Slightly Delayed     | 41             | 120.00    |
| Seriously Delayed    | 35             | 720.00    |
| Other                | 19             | 0.00      |
| Failed               | 17             | 0.00      |

### Query 6 — Delivery Filter by Warehouse and Date Range

Purpose: Retrieve all deliveries dispatched from a specific warehouse within a date range. This is the most frequently executed operational query, used for warehouse reports and daily planning.

Techniques demonstrated: Equality filter combined with range filter, index-friendly conditions (no DATE() function wrapper on the column), supports EXPLAIN before/after index analysis.

```
-- -----  
-- QUERY 6  
-- Performance: filter deliveries by warehouse_id + date range.  
-- Uses index-friendly range condition (no DATE() wrapper on column).  
-- The composite index is created in SECTION 8.  
-- -----  
  
SELECT delivery_id, shipment_id, driver_id, status, delay_minutes, actual_date  
FROM deliveries  
WHERE warehouse_id = 1  
      AND pickup_date >= '2025-09-01'  
-    AND pickup_date <  '2026-01-01';
```

[INSERT QUERY 6 OUTPUT TABLE]

| delivery_id | shipment_id | driver_id | status    | delay_minutes | actual_date         |
|-------------|-------------|-----------|-----------|---------------|---------------------|
| 1           | 1           | 1         | delivered | 0             | 2025-10-16 00:00:00 |
| 2           | 2           | 2         | delivered | 0             | 2025-10-03 12:00:00 |
| 3           | 3           | 3         | delivered | 0             | 2025-09-21 00:00:00 |
| 4           | 4           | 4         | delivered | 0             | 2025-09-08 12:00:00 |
| NULL        | NULL        | NULL      | NULL      | NULL          | NULL                |

## 7. Relational Algebra

The following extended relational algebra expressions correspond to six implemented SQL queries. The notation used is:

- $\sigma$  (Sigma) — Selection (equivalent to WHERE / HAVING)
- $\pi$  (Pi) — Projection (equivalent to SELECT columns)
- $\bowtie$  (Bowtie) — Theta Join (equivalent to JOIN ON)
- $\gamma$  (Gamma) — Grouping and Aggregation (equivalent to GROUP BY with aggregates)
- $\tau$  (Tau) — Sorting (equivalent to ORDER BY)

### Query 1 — Top 5 Drivers

Logic: Join deliveries with drivers, filter for on-time records, group by driver, aggregate counts and average duration, sort, and limit.

```
Limit5(  $\tau$  on_time DESC, avg_dur ASC (  $\pi$  driver_id, driver_name, on_time, avg_dur  
(  $\gamma$  driver_id,name; COUNT(*) $\rightarrow$ on_time, AVG(Duration) $\rightarrow$ avg_dur (  $\sigma$   
status='delivered'  $\wedge$  delay_minutes=0  $\wedge$  actual_date $\neq$ NULL ( Deliveries  $\bowtie$   
D1.driver_id=D.driver_id Drivers ) ) ) )
```

### Query 2 — Warehouses Above Average Delay

Logic: Calculate global average first, then filter warehouse groups whose average exceeds it.

Step 1 – Global average: GlobalAvg =  $\gamma$  AVG(delay\_minutes) (Deliveries)

Step 2 – Main query:  $\tau$  avg\_delay DESC (  $\sigma$  avg\_delay > GlobalAvg (  $\gamma$  w.id,w.name;  
AVG(delay\_minutes) $\rightarrow$ avg\_delay, COUNT(\*) $\rightarrow$ total ( Deliveries  $\bowtie$   
D1.warehouse\_id=W.warehouse\_id Warehouses ) ) )

### Query 3 — Delayed Routes

Logic: Join deliveries, routes, and cities (twice). Apply conditional logic inside aggregation. Filter routes where delayed count exceeds on-time count.

Cond\_d = CASE WHEN status='delayed' THEN 1 ELSE 0

Cond\_o = CASE WHEN status='delivered'  $\wedge$  delay\_minutes=0 THEN 1 ELSE 0

```
 $\tau$  delayed_count DESC (  $\sigma$  delayed_count > on_time_count (  $\gamma$  r.id,co.name,cd.name;  
SUM(Cond_d) $\rightarrow$ delayed_count, SUM(Cond_o) $\rightarrow$ on_time_count ( Deliveries  $\bowtie$  Routes  $\bowtie$   
co.id=r.origin_id Cities_origin  $\bowtie$  cd.id=r.dest_id Cities_dest ) ) )
```

### Query 4 — Rank Cities by Volume

Logic: Join shipments, warehouses, and cities; group by city; apply window function for ranking.

```

τ volume_rank ASC ( π city_id, city_name, total_volume,
RANK()OVER(...)→volume_rank ( γ c.city_id,c.city_name;
SUM(volume_m3)→total_volume ( Shipments ▷◁ Warehouses ▷◁ Cities ) ) )

```

## Query 5 — Classify Deliveries

Logic: Project a performance category column using CASE logic, group by it, aggregate.

```

τ delivery_count DESC ( γ PerfCat; COUNT(*)→delivery_count,
AVG(delay_minutes)→avg_delay ( π CASE(...)→PerfCat, delay_minutes (Deliveries) )
)

```

## Query 6 — Simple Range Filter

Logic: Apply selection predicates on warehouse\_id and pickup\_date range, then project the required columns.

```

π delivery_id, shipment_id, driver_id, status, delay_minutes, actual_date ( σ
warehouse_id=1 ∧ pickup_date≥'2025-09-01' ∧ pickup_date<'2026-01-01'
(Deliveries) )

```

## 8. SQL Views

Two analytical views are defined to simplify reporting. Each view encapsulates a complex multi-table query and presents a clean summary that can be queried like a table.

### View 1 — driver\_performance\_view

Purpose: Provides a comprehensive performance summary for every driver, including total deliveries, on-time count, delayed count, failed count, and average delay. Useful for driver reviews, HR decisions, and SLA reporting.

```
-- ----- VIEW 1: driver_performance_view -----
CREATE OR REPLACE VIEW driver_performance_view AS
SELECT
    d.driver_id,
    CONCAT(d.first_name, ' ', d.last_name) AS driver_name,
    d.rating,
    COUNT(dl.delivery_id) AS total_deliveries,
    SUM(CASE WHEN dl.status = 'delivered' AND dl.delay_minutes = 0
        THEN 1 ELSE 0 END) AS on_time_deliveries,
    SUM(CASE WHEN dl.status = 'delayed' THEN 1 ELSE 0 END) AS delayed_deliveries,
    SUM(CASE WHEN dl.status = 'failed' THEN 1 ELSE 0 END) AS failed_deliveries,
    ROUND(AVG(dl.delay_minutes), 2) AS avg_delay_minutes
FROM drivers d
LEFT JOIN deliveries dl ON dl.driver_id = d.driver_id
GROUP BY d.driver_id, d.first_name, d.last_name, d.rating;
```

| driver_id | driver_name          | rating | total_deliveries | on_time_deliveries | delayed_deliveries | failed_deliveries | avg_delay_minutes |
|-----------|----------------------|--------|------------------|--------------------|--------------------|-------------------|-------------------|
| 1         | Aibek Nursultanov    | 4.85   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 61        | Sultan Amangeldy     | 4.37   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 3         | Erlan Smagulov       | 4.90   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 59        | Orynbasar Moldabekov | 4.66   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 5         | Nurzhan Abdykaimov   | 4.80   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 57        | Kazbek Toktarov      | 4.91   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 55        | Ersain Taipov        | 4.43   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 52        | Almas Tursynov       | 4.50   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 48        | Olzhas Meiramov      | 4.84   | 5                | 5                  | 0                  | 0                 | 0.00              |
| 46        | Kuanysh Baizhanov    | 4.79   | 5                | 5                  | 0                  | 0                 | 0.00              |

### View 2 — warehouse\_performance\_view

Purpose: Aggregates delivery performance, revenue, and failure rates per warehouse. Allows operations managers to compare warehouses on key metrics without writing complex joins each time.

```

-- ----- VIEW 2: warehouse_performance_view -----
CREATE OR REPLACE VIEW warehouse_performance_view AS
SELECT
    w.warehouse_id,
    w.warehouse_name,
    c.city_name,
    COUNT(dl.delivery_id)                                AS total_deliveries,
    ROUND(AVG(dl.delay_minutes), 2)                     AS avg_delay_minutes,
    SUM(CASE WHEN dl.status = 'delivered' THEN 1 ELSE 0 END) AS delivered_count,
    SUM(CASE WHEN dl.status = 'failed' THEN 1 ELSE 0 END)   AS failed_count,
    ROUND(SUM(dl.delivery_cost), 2)                     AS total_revenue
FROM warehouses w
JOIN cities c ON c.city_id = w.city_id
LEFT JOIN deliveries dl ON dl.warehouse_id = w.warehouse_id
GROUP BY w.warehouse_id, w.warehouse_name, c.city_name;

```

| warehouse_id | warehouse_name          | city_name  | total_deliveries | avg_delay_minutes | delivered_count | failed_count | total_revenue |
|--------------|-------------------------|------------|------------------|-------------------|-----------------|--------------|---------------|
| 49           | Samarkand Partner Depot | Samarkand  | 7                | 120.00            | 4               | 1            | 123060.00     |
| 48           | Astana Expo Depot       | Astana     | 7                | 120.00            | 5               | 0            | 121247.00     |
| 47           | Almaty Tech Park Hub    | Almaty     | 7                | 120.00            | 4               | 0            | 119434.00     |
| 43           | Beineu Transit          | Beineu     | 7                | 120.00            | 4               | 1            | 112182.00     |
| 42           | Dostyk Rail Depot       | Dostyk     | 7                | 120.00            | 4               | 0            | 110369.00     |
| 45           | Aralsk Depot            | Aralsk     | 7                | 102.86            | 6               | 0            | 115808.00     |
| 50           | Kazan Transit Hub       | Kazan      | 7                | 17.14             | 6               | 0            | 124873.00     |
| 46           | Kandyagash Transit      | Kandyagash | 7                | 17.14             | 5               | 1            | 117621.00     |
| 44           | Shu Junction Depot      | Shu        | 7                | 17.14             | 5               | 0            | 113995.00     |
| 41           | Saryagash Border Hub    | Saryagash  | 7                | 17.14             | 6               | 0            | 108556.00     |



## 9. Trigger

### trg\_delivery\_status\_audit

Event: AFTER UPDATE on the deliveries table.

Business purpose: Every time a delivery's status changes (e.g. pending → in\_transit → delivered), the trigger automatically writes a record to delivery\_status\_log capturing the old status, the new status, the exact timestamp, and the database user who made the change. This creates a complete, tamper-evident audit trail that can be used for SLA dispute resolution, driver accountability, and operational analytics.

```
DELIMITER //
CREATE TRIGGER trg_delivery_status_audit
AFTER UPDATE ON deliveries
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        INSERT INTO delivery_status_log (delivery_id, old_status, new_status, changed_by)
        VALUES (NEW.delivery_id, OLD.status, NEW.status, CURRENT_USER());
    END IF;
END//
DELIMITER ;
```

Trigger demonstration:

```
-- Trigger demo:
UPDATE deliveries SET status = 'in_transit' WHERE delivery_id = 1;
UPDATE deliveries SET status = 'delivered' WHERE delivery_id = 1;
SELECT * FROM delivery_status_log WHERE delivery_id = 1;
```

Result after demonstration:

| log_id | delivery_id | old_status | new_status | changed_at          | changed_by     |
|--------|-------------|------------|------------|---------------------|----------------|
| 1      | 1           | NULL       | delivered  | 2026-05-18 19:39:49 | system_init    |
| 64     | 1           | delivered  | in_transit | 2026-05-18 19:56:24 | root@localhost |
| 65     | 1           | in_transit | delivered  | 2026-05-18 19:56:24 | root@localhost |
| NULL   | NULL        | NULL       | NULL       | NULL                | NULL           |

The trigger only fires when OLD.status <> NEW.status, preventing spurious log entries when other delivery columns are updated without a status change.

## 10. Transactions

Two transaction scenarios are demonstrated: one with COMMIT (successful atomic operation) and one with ROLLBACK (deliberate cancellation). Both illustrate the ACID properties of InnoDB.

### 10A — COMMIT Scenario

Business scenario: Creating a new shipment and its delivery record must be atomic. If either INSERT fails, neither row should be saved. This prevents orphaned shipments with no delivery, or deliveries with no matching shipment.

```
START TRANSACTION;

INSERT INTO shipments (client_id, warehouse_id, weight_kg, volume_m3, description, priority)
VALUES (1, 1, 250.00, 3.10, 'Transaction demo shipment', 'high');

SET @new_shipment_id = LAST_INSERT_ID();

INSERT INTO deliveries
  (shipment_id, driver_id, vehicle_id, route_id, warehouse_id,
  ~ status, pickup_date, expected_date, delivery_cost)
VALUES
  ~ (@new_shipment_id, 1, 1, 1, 1,
  ~ 'pending', '2026-05-13 09:00:00', '2026-05-14 09:00:00', 45000.00);

COMMIT;

-- After COMMIT, both rows are permanently saved.
-- Verify: this SELECT must return 1 row.
SELECT * FROM shipments WHERE description = 'Transaction demo shipment';
```

After COMMIT, both the shipment and its delivery are permanently saved. A SELECT query on description = 'Transaction demo shipment' returns exactly one row.

| shipment_id | client_id | warehouse_id | weight_kg | volume_m3 | description               | created_at          | priority |
|-------------|-----------|--------------|-----------|-----------|---------------------------|---------------------|----------|
| 512         | 1         | 1            | 250.00    | 3.10      | Transaction demo shipment | 2026-05-18 19:57:31 | high     |
| NULL        | NULL      | NULL         | NULL      | NULL      | NULL                      | NULL                | NULL     |

### 10B — ROLLBACK Scenario

Business scenario: A shipment INSERT begins inside a transaction, but an error is detected before the delivery INSERT can be completed. ROLLBACK undoes the shipment INSERT, leaving the database in its original clean state.

```
START TRANSACTION;
```

```
-- Step 1: insert a shipment inside the transaction
```

```
INSERT INTO shipments (client_id, warehouse_id, weight_kg, volume_m3, description, priority)
VALUES (2, 1, 100.00, 1.20, 'Rollback demo shipment', 'normal');
```

```
SET @rollback_shipment_id = LAST_INSERT_ID();
```

```
-- Step 2: this SELECT shows the row EXISTS inside the transaction
```

```
-- (the row is visible to the current session but not yet committed)
```

```
SELECT * FROM shipments WHERE shipment_id = @rollback_shipment_id;
```

```
-- Step 3: something went wrong (e.g. delivery creation would fail),
```

```
-- so we roll back the entire transaction
```

ROLLBACK;

```
-- Step 4: this SELECT returns EMPTY result – the shipment is gone.
```

```
-- ROLLBACK undid the INSERT as if it never happened.
```

```
SELECT * FROM shipments WHERE shipment_id = @rollback_shipment_id;
```

This demonstrates MySQL InnoDB's full support for ACID transactions: the shipment is atomically either committed or rolled back, with no partial state visible to other sessions.

[illegible]

| shipment_id | client_id | warehouse_id | weight_kg | volume_m3 | description | created_at | priority |
|-------------|-----------|--------------|-----------|-----------|-------------|------------|----------|
| NULL        | NULL      | NULL         | NULL      | NULL      | NULL        | NULL       | NULL     |
|             |           |              |           |           |             |            |          |

## 11. Indexing and Query Optimization

Two composite indexes are created to optimize the most performance-sensitive queries. For each index, an EXPLAIN analysis is performed before and after index creation to document the improvement.

### Index 1 — idx\_deliveries\_wh\_pickup

Supports: Query 6 — filtering deliveries by warehouse and date range.

-- Create composite index:

```
CREATE INDEX idx_deliveries_wh_pickup
ON deliveries (warehouse_id, pickup_date);
```

Strategy: The index is composite with warehouse\_id first (equality filter) and pickup\_date second (range filter). This is the optimal B-Tree index column order: equality conditions come before range conditions. No DATE() wrapper is used on the pickup\_date column, ensuring the index is used for the range scan.

| Metric        | Before Index          | After Index              |
|---------------|-----------------------|--------------------------|
| Access type   | ALL (full table scan) | range (index range scan) |
| Rows examined | ~350                  | ~8                       |
| Key used      | NULL                  | idx_deliveries_wh_pickup |
| Extra         | Using where           | Using index condition    |

Result: The optimizer drops from scanning all 350 rows to examining approximately 8 rows — a ~43x improvement in rows read.

### Index 2 — idx\_deliveries\_route\_status\_delay

Supports: Query 3 — finding routes where delayed deliveries outnumber on-time deliveries.

-- Create composite index:

```
CREATE INDEX idx_deliveries_route_status_delay
ON deliveries (route_id, status, delay_minutes);
```

Strategy: The composite index covers route\_id (used in the JOIN), status (used in CASE WHEN conditions), and delay\_minutes (used in the second CASE WHEN). Because all three columns are in the index, MySQL can perform a covering index scan — reading data directly from the index without accessing the base table rows, which eliminates disk I/O for the most expensive part of Query 3.

| Metric      | Before Index          | After Index                 |
|-------------|-----------------------|-----------------------------|
| Access type | ALL (full table scan) | index (covering index scan) |

|               |                                              |                                   |
|---------------|----------------------------------------------|-----------------------------------|
| Rows examined | ~350                                         | ~350 (but from index, not table)  |
| Key used      | NULL                                         | idx_deliveries_route_status_delay |
| Extra         | Using where; Using temporary; Using filesort | Using index                       |

The key improvement is the transition from Using temporary; Using filesort to Using index. The optimizer can now resolve the GROUP BY and conditional SUM entirely from the index structure, eliminating the sort and temp table overhead.

```

SELECT '--- EXPLAIN ANALYZE: BEFORE INDEX (Simulated Table Scan) ---' AS Test_Phase LIMIT 0, 1000  1  0.0019 sec / 0.0000031 sec
EXPLAIN ANALYZE SELECT delivery_id, shipment_id, driver_id, status, delay_minutes, actual_date FRO...  1  0.014 sec / 0.0000041 sec
SELECT '--- EXPLAIN ANALYZE: AFTER INDEX (Index Lookup) ---' AS Test_Phase LIMIT 0, 1000          1  0.00023 sec / 0.0000021 sec
EXPLAIN ANALYZE SELECT delivery_id, shipment_id, driver_id, status, delay_minutes, actual_date FRO...  1  0.0010 sec / 0.0000021 sec
SELECT '--- EXPLAIN ANALYZE: BEFORE INDEX 2 (Simulated) ---' AS Test_Phase LIMIT 0, 1000        1  0.00014 sec / 0.0000019 sec
EXPLAIN ANALYZE SELECT  r.route_id,  co.city_name AS origin_city,  cd.city_name AS dest_city,...  1  0.0047 sec / 0.0000029 sec
SELECT '--- EXPLAIN ANALYZE: AFTER INDEX 2 ---' AS Test_Phase LIMIT 0, 1000                  1  0.00012 sec / 0.00000095 sec

```

## 12. Dataset Summary

The LogiTrack database is populated with realistic, logically connected data meeting and exceeding the minimum data volume requirements.

| Table               | Row Count | Type          | Generation Method                                            |
|---------------------|-----------|---------------|--------------------------------------------------------------|
| cities              | 60        | Reference     | Manual — real Kazakhstani and regional cities                |
| warehouses          | 50        | Reference     | Manual — named warehouses in real city addresses             |
| vehicle_types       | 6         | Reference     | Manual — 6 categories from motorcycle to heavy freight       |
| vehicles            | 80        | Reference     | Manual — unique plate numbers, assigned to warehouses        |
| drivers             | 70        | Reference     | Manual — Kazakhstani names, phones, license numbers          |
| clients             | 80        | Reference     | Manual — real business names, unique emails                  |
| routes              | 80        | Reference     | Manual — bidirectional real-world distances                  |
| shipments           | 350       | Transactional | WITH RECURSIVE generator — diverse priorities and categories |
| deliveries          | 350       | Transactional | Derived from shipments — realistic status distribution       |
| delivery_status_log | 50        | Audit         | Initial seed from system_init trigger simulation             |

### Delivery Status Distribution

The 350 delivery records follow a realistic operational distribution:

| Status              | Approx. Count | Approx. % | Trigger Condition                |
|---------------------|---------------|-----------|----------------------------------|
| delivered (on time) | ~220          | ~63%      | Default (all other shipment_ids) |

|                              |     |      |                                |
|------------------------------|-----|------|--------------------------------|
| delayed                      | ~75 | ~21% | shipment_id % 7 = 0 or % 9 = 0 |
| failed                       | ~17 | ~5%  | shipment_id % 20 = 0           |
| in_transit                   | ~20 | ~6%  | shipment_id % 17 = 0           |
| delivered (slightly delayed) | ~18 | ~5%  | delay = 120 min                |

The data supports all six analytical queries with meaningful, non-trivial results including cases where warehouses exceed average delay (Query 2), routes with more delays than successes (Query 3), and a spread of performance categories (Query 5).

## 13. Business Logic and Design Decisions

### 13.1 Delivery-Shipment 1:1 Relationship

The UNIQUE constraint on `deliveries.shipment_id` enforces that every shipment has at most one delivery attempt recorded in this table. This reflects the business rule that each shipment order results in a single dispatch event. Re-delivery scenarios would require a new shipment to be created. This design keeps the deliveries table clean and prevents double-dispatching.

### 13.2 Route Symmetry

Routes are defined directionally (Almaty → Astana is a separate row from Astana → Almaty). This allows different distances or durations for each direction, which is realistic for road networks with one-way highways, traffic patterns, and mountain passes.

### 13.3 Delay in Minutes

Delay is stored as an integer number of minutes rather than a duration type, enabling arithmetic comparisons in CASE WHEN (e.g. `delay > 180` for 'Seriously Delayed'). This is more portable and query-friendly than interval types.

### 13.4 Audit Log Design

The `delivery_status_log` table stores `old_status` as nullable (NULL for the initial insertion, when there is no previous state). This allows the system to capture the full history from the very first status assignment without requiring a sentinel value.

### 13.5 Driver Rating

Driver ratings (DECIMAL 3,2) range from 1.00 to 5.00 and default to 5.00 on hire. This creates a downward-only starting point where ratings decrease with poor performance, which is a common logistics company practice.

### 13.6 Index-Friendly Query Design

Query 6 deliberately avoids wrapping `pickup_date` in a `DATE()` function call. Writing `WHERE pickup_date >= '2025-09-01'` instead of `WHERE DATE(pickup_date) >= '2025-09-01'` ensures the composite B-Tree index can be used for range scanning. Wrapping a column in a function prevents index usage because MySQL cannot predict the function output for each index entry.



## 14. Conclusion

The LogiTrack database project successfully fulfills all requirements of the Final Exam specification:

| Requirement                   | Target            | Achieved                                                      |
|-------------------------------|-------------------|---------------------------------------------------------------|
| Tables                        | 8–10              | 10 ✓                                                          |
| Relationships (FK)            | 15+               | 16 ✓                                                          |
| Normal Form                   | 3NF               | 3NF ✓                                                         |
| Constraint types              | 6 types           | PK, FK, UNIQUE, NOT NULL, DEFAULT, CHECK ✓                    |
| CHECK constraints             | 3+ meaningful     | 10+ CHECK constraints ✓                                       |
| Main transactional table rows | 300+              | 350 deliveries, 350 shipments ✓                               |
| Reference table rows          | 50+               | 60 cities, 50 warehouses, 80 clients, 70 drivers, 80 routes ✓ |
| Analytical SQL queries        | 6                 | 6 ✓ (JOIN, aggregation, subquery, window, CASE, EXPLAIN)      |
| Views                         | 2+                | 2 ✓                                                           |
| Trigger                       | 1+                | 1 ✓ (status audit)                                            |
| Transactions                  | COMMIT + ROLLBACK | Both demonstrated ✓                                           |
| Relational Algebra            | 3+ expressions    | 6 expressions ✓                                               |
| Indexes with EXPLAIN          | 2+                | 2 composite indexes with before/after EXPLAIN ✓               |

The project demonstrates a production-grade relational database design with realistic data, advanced SQL analytics, and proper performance optimization. All components are modular, well-documented, and executable in sequence on a standard MySQL 8.0 server.